New Horizons
Centro de Capacitación

Práctica: Caching en Memoria y Optimización de Consultas en EF Core

Paso a paso — Configurar IMemoryCache, expiración deslizante y consultas LINQ optimizadas

Curso	Desarrollo Full Stack con C# y Angular
Sesión	Sesión 17 — Optimización y Rendimiento en Backend y Frontend
Docente	Edwin Barrientos
Duración estimada	75–90 minutos
Nivel	Básico (guía principal de la sesión)
Entorno	.NET 8 SDK, Visual Studio 2022 / VS Code, Angular 18 Standalone, Bootstrap 5

1. Objetivos de la Práctica

- Registrar AddMemoryCache en el contenedor de dependencias de ASP.NET Core.
- Implementar almacenamiento en caché de centros de costo y roles con expiración.
- Optimizar consultas con AsNoTracking y proyecciones directas en SolicitudRepository.
- Medir la reducción en los tiempos de respuesta de la API mediante Postman.
- Invalidar la caché cuando se modifiquen los catálogos del sistema.

2. Desarrollo Paso a Paso

Paso 1 — Configurar IMemoryCache en Application

Inyectamos IMemoryCache en el servicio de catálogos:

src/SolicitudesApp.Application/Services/CatalogoService.cs

```
public class CatalogoService : ICatalogoService
{
    private readonly IMemoryCache _cache;
    private readonly SolicitudesDbContext _db;

    public CatalogoService(IMemoryCache cache, SolicitudesDbContext db)
    {
        _cache = cache;
        _db = db;
    }

    public async Task<List<CentroCosto>> ObtenerCentrosCostoAsync()
    {
        return await _cache.GetOrCreateAsync("cat_centros_costo", entry => {
            entry.SlidingExpiration = TimeSpan.FromHours(2);
            return _db.CentrosCosto.AsNoTracking().ToListAsync();
        })!;
    }
}
```

> **Validación de la Práctica:** Asegúrese de compilar y ejecutar las pruebas para validar que los endpoints y vistas operen conforme a los estándares corporativos solicitados.